

Образовательная платформа «Командный зачёт»

Материал для презентации: технологии, функции, архитектура, адаптивность

1. Что мы создали (MVP)

Веб-приложение с личным кабинетом студента и серверной частью. Цель — снизить стресс перед сессией и автоматизировать командную работу в УрФУ.

- Авторизация — регистрация, вход, JWT-сессия
- Профиль — фото, имя, группа, лига, баллы, рейтинг, достижения
- Команда — создание/поиск, заявки, invite (QR/ссылка), состав, КРК
- Check-in — еженедельный отчёт команды (капитан)
- Спасение — запрос помощи другой команде + бонусные баллы
- Голосование — анонимная оценка вклада участников (1–5)
- Рейтинг — лидерборды команд и студентов, фильтры, карточки
- События — календарь, создание событий, лента активности и новостей
- Админка (API) — начисление баллов, пересчёт КРК, модерация

В планах / заглушки: OAuth УрФУ (Active Directory), импорт оценок с учебного портала, «Биржа знаний» и «Задания» в UI (часть API уже на backend).

2. Технологический стек

Frontend

- TypeScript — строгая типизация
- Vite 8 — сборка и dev-сервер
- Vanilla TS + модульная архитектура (компоненты, pages, services)
- CSS-модули: mobile, profile, team, events, auth, theme
- QRCode — QR для приглашений и событий

Примечание: в README указан React/Next.js; в текущем коде — Vite + TypeScript без React.

Backend

- ASP.NET Core 8 (C#) — REST API
- Entity Framework Core — ORM
- PostgreSQL 16 — база данных
- JWT Bearer — авторизация
- Swagger — документация API (dev)
- PasswordHasher — хеширование паролей

DevOps

- Docker + Docker Compose — API + PostgreSQL
- Multi-stage Dockerfile — frontend → wwwroot backend → publish API
- Nginx — reverse proxy (prod-конфиг)
- Запуск: docker compose up --build → http://localhost:8080

3. Архитектура

Браузер (Desktop / Tablet / Mobile) → Frontend (static в wwwroot ASP.NET) → REST + JWT → ASP.NET Core API →

Паттерны и подходы

- SPA: один start.ts, переключение экранов без перезагрузки
- AppBridge — связь страниц (TeamPage, RatingPage) с бизнес-логикой
- AppState — централизованное состояние UI
- localStorage — JWT и часть UI-состояния
- KrkCached — кэш КРК в таблице команд

4. Бизнес-логика: формула КРК

$KPK = (\text{Базовый рейтинг} \times 0.6) + (\text{Сплочённость} \times 0.3) + (\text{Челленджи} \times 0.1)$

Шкала 0–10. Реализация: KrkCalculationService.cs, веса KrkOptions.

- Базовый рейтинг (60%) — Team.Score, нормализация 0–10 относительно max
- Сплочённость (30%) — среднее голосов 1–5 → 0–10
- Челленджи (10%) — одобренные челленджи → 0–10 относительно max bonus
- Спасение — HelpRequest между командами, бонусные баллы
- Голосование — анонимные оценки участников
- Check-in — еженедельный отчёт капитана
- Роли: Student, Captain, Admin

5. Backend API — контроллеры

- AuthController — register, login, me
- ProfileController — GET/PUT профиль
- TeamsController — команды, join, invite, заявки
- RatingsController — рейтинг команд и пользователей
- VotesController — голосование
- CheckInsController — check-in
- HelpRequestsController — спасение
- ChallengesController — челленджи
- EventsController — календарь
- NewsController — новости
- ActivityFeedController — лента активности
- AchievementsController — достижения
- KnowledgePostsController — биржа знаний (API)
- GroupsController — академические группы
- AdminController — баллы, пересчёт КРК, модерация

Безопасность: JWT Authorization Bearer, ролевая модель, CORS.

6. Frontend — разделы и компоненты

Разделы dashboard

- Профиль — hero-карточка, статистика, достижения
- Команда — topbar, карусель участников, история активности, check-in, спасение
- Рейтинг — таблицы, фильтры, карточки профилей
- События — календарь, создание событий, активность / новости
- Задания — пункт меню disabled, в разработке

UI-функции

- Модалки: профиль, команда, голосование, заявки, спасение, check-in, события
- Просмотр заявки: профиль кандидата + dock «К ЗАЯВКАМ / ОТКЛОНИТЬ / ПРИНЯТЬ»
- Карусель участников — стрелки только при переполнении
- QR-коды для invite и событий
- Анимации auth и мобильное swipe-меню

7. База данных — сущности

- User, Team, TeamJoinRequest, Vote, CheckIn, HelpRequest
 - Challenge + TeamChallengeProgress, CalendarEvent, NewsItem
 - ActivityFeedItem, Achievement, KnowledgePost, Group
- Миграции EF Core, PostgreSQL в Docker volume.

8. Адаптивность на разных устройствах

Breakpoints

- Desktop (>900px) — sidebar + dashboard
- Tablet (768–1024px) — горизонтальный swipe auth
- Mobile (≤767px) — hamburger-меню, swipe, компактная вёрстка
- ≤520px / ≤380px — дополнительное ужатие карточек и модалок

Desktop

- Боковое меню: Профиль, Команда, Рейтинг, События, Настройки, Выход
- Профиль: grid photo + stats, блок достижений
- Команда: topbar, карусель, фиксированная история активности

Mobile / Tablet

- Скрытый sidebar → гамбургер + backdrop + swipe
- Auth-шторка со swipe между входом и регистрацией
- Touch-friendly: pointer events, ResizeObserver для карусели
- clamp(), flexbox, grid, glassmorphism (backdrop-filter)

9. Команда проекта

- Егор Габов — TL, Frontend, DBA — архитектура, БД
- Полина Желнова — UX/UI — User Flow, макеты
- Никита Малышкин — Frontend — вёрстка, лидерборды
- Егор Шадрин — Backend — API, КРК, команды
- Валерий Салимгареев — Backend, DevOps — CI/CD, auth, деплой

10. Требования

- Отклик UI/API — менее 2 секунд
- Uptime — не менее 99%
- Безопасность — JWT, роли, защита персональных данных
- Масштабируемость — SaaS для институтов УрФУ

11. Roadmap

- Фаза 1 — проектирование (дизайн, БД, репозиторий)
- Фаза 2 — Core: ЛК, команды, КРК, лидерборды

- Фаза 3 — биржа знаний, челленджи, спасение, импорт с портала

- Фаза 4 — QA, админ-панель, пилотный запуск

Текущий статус: MVP с профилем, командой, рейтингом, событиями; широкий backend API; OAuth УрФУ и импорт портала — заглушки.

12. Тезисы для слайдов

- Слайд 1: Проблема: стресс перед сессией, сложно организовать команду

- Слайд 2: Решение: геймифицированная платформа «Командный зачёт»

- Слайд 3: Стек: TypeScript + Vite, ASP.NET Core 8, PostgreSQL, Docker

- Слайд 4: Архитектура: SPA + REST API + JWT

- Слайд 5: КРК: формула 60/30/10, автоматический пересчёт

- Слайд 6: Механики: команды, check-in, спасение, голосование, челленджи

- Слайд 7: UI: 4 раздела dashboard, модалки, QR, адаптив

- Слайд 8: Устройства: desktop sidebar, mobile swipe-меню

- Слайд 9: Команда и roadmap

- Слайд 10: Демо: localhost:8080 после docker compose up --build